

# When Agents Act on Web3: An Attack-Surface Survey of MCP, Skills, and Tool Calling

Rabimba Karanjai<sup>1,3</sup>, Yang Lu<sup>1</sup>, Nour Diallo<sup>1</sup>, Wujie Xiong<sup>2</sup>, Lei Xu<sup>2</sup>, and Weidong (Larry) Shi<sup>1</sup>

<sup>1</sup>University of Houston, USA, <sup>2</sup>Kent State University, USA, <sup>3</sup>PayPal AI Labs, USA

AI agents increasingly act rather than merely read: across the Model Context Protocol (MCP) ecosystem, the share of deployed tools that modify external state has risen from 27% to 65% of tool use. When agents exercise this authority on public blockchains through MCP, skills, and tool calling, the consequences of an attack are governed by the blockchain execution layer rather than by conventional software assumptions. This survey argues that four properties of that layer (irreversibility, signing authority, continuous autonomy, and sequence-level composition) qualitatively change the threat model, turning the recoverable failures of generic agent security into a standing, irreversible loss. We organize the fragmented MCP-security literature into an attack-surface taxonomy, then contribute a Web3 risk-mapping matrix that ties each attack class to its amplified impact, the responsible amplifiers, a representative mitigation, and the residual gap. We synthesize defenses, including emerging blockchain-based mechanisms, and find them improving but insufficient: measured protections stop fewer than 30% of attacks, and model-level safety refuses fewer than 3%. We close by positioning the work against adjacent surveys and deriving a research agenda from the matrix's open cells.

*Keywords:* Model Context Protocol, AI agents, tool calling, agentic AI, Web3, blockchain security, smart-contract security, attack surface, supply-chain security, LLM security

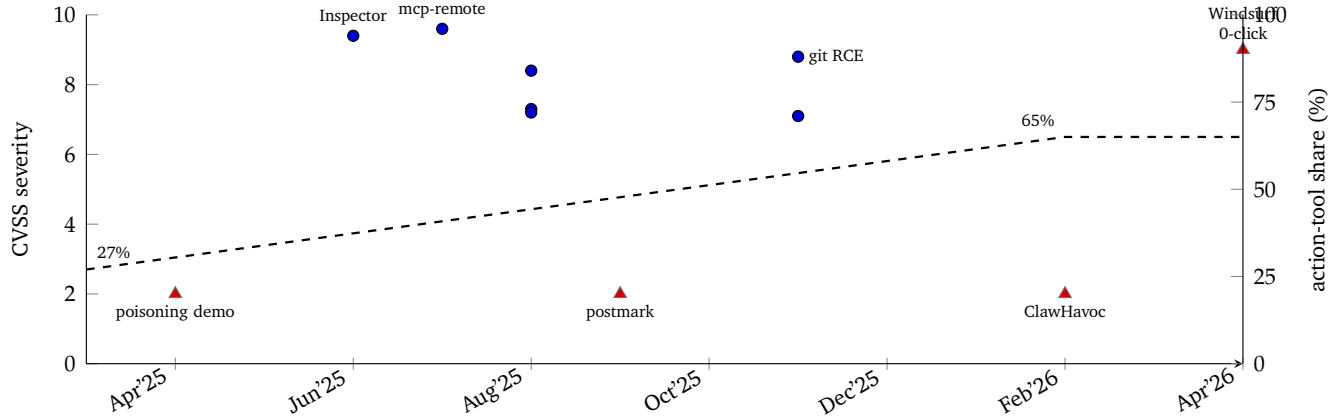
## 1. Introduction

AI agents have crossed a threshold from reading to acting. Across a sixteen-month window of the Model Context Protocol (MCP) ecosystem, the share of deployed agent tools that take *actions*, modifying external state rather than only observing it, rose from 27% to 65% of tool use, while the number of distinct tools grew from roughly 5,000 to over 177,000 [Stein \(2026\)](#). Among the domains where this shift is fastest are financial ones, where an agent's action is a transaction.

MCP, introduced in late 2024 and since placed under open governance, has become the dominant standard for connecting agents to external tools [Anthropic \(2024\)](#), [Linux Foundation \(2025\)](#). Its rapid adoption has been matched by a rapid accumulation of security failures. Within roughly a year, confirmed vulnerabilities progressed from the first demonstration of tool poisoning to a cluster of high-severity remote code execution and authentication flaws, and to the first documented malicious MCP server (Figure 1).

These two trends (agents gaining authority to act, and the tools that grant that authority proving insecure) intersect most sharply on public blockchains. When an agent uses MCP, skills, or tool-calling to act on a Web3 system, the consequences of a successful attack are governed not by conventional software assumptions but by the blockchain execution layer. This paper's thesis is that four properties of that layer (**irreversibility**, **signing authority**, **continuous autonomy**, and **sequence-level composition**) fundamentally change the threat model, turning the recoverable failures of generic agent security into standing, irreversible loss.

We treat the *usage-to-risk* direction as primary: our subject is the attack surface that opens when agents



**Figure 1:** Escalation of confirmed MCP/tool-calling security disclosures (Apr 2025–Apr 2026) against ecosystem growth. Circles: CVEs (height = CVSS). Triangles: documented incidents. Dashed line: share of *action* tools rising 27%→65% as total tools grew 5,000→177,436.

act on Web3, not the separate question of using blockchains to secure agents, which we address only as a class of mitigations.

This paper makes five contributions:

- A **web3 risk-mapping matrix** (Table 2) that ties each attack class to its amplified Web3 impact, the amplifiers responsible, a representative mitigation, and the residual gap, exposing where current defenses are inadequate.
- A **four-amplifier framework** that explains *why* the blockchain execution layer changes the threat model, distinguishing Web3 agent risk from generic agent risk.
- A **synthesis of defenses with honest coverage gaps**, including a moderate treatment of blockchain-based defenses, mapped to the attack surface.
- An **attack-surface taxonomy** (Table 1) that organizes the fragmented MCP-security literature along surface and lifecycle-stage axes, serving as the scaffolding for the analysis above.
- A **scoped positioning** against adjacent surveys and a **research agenda** drawn from the matrix’s residual gaps.

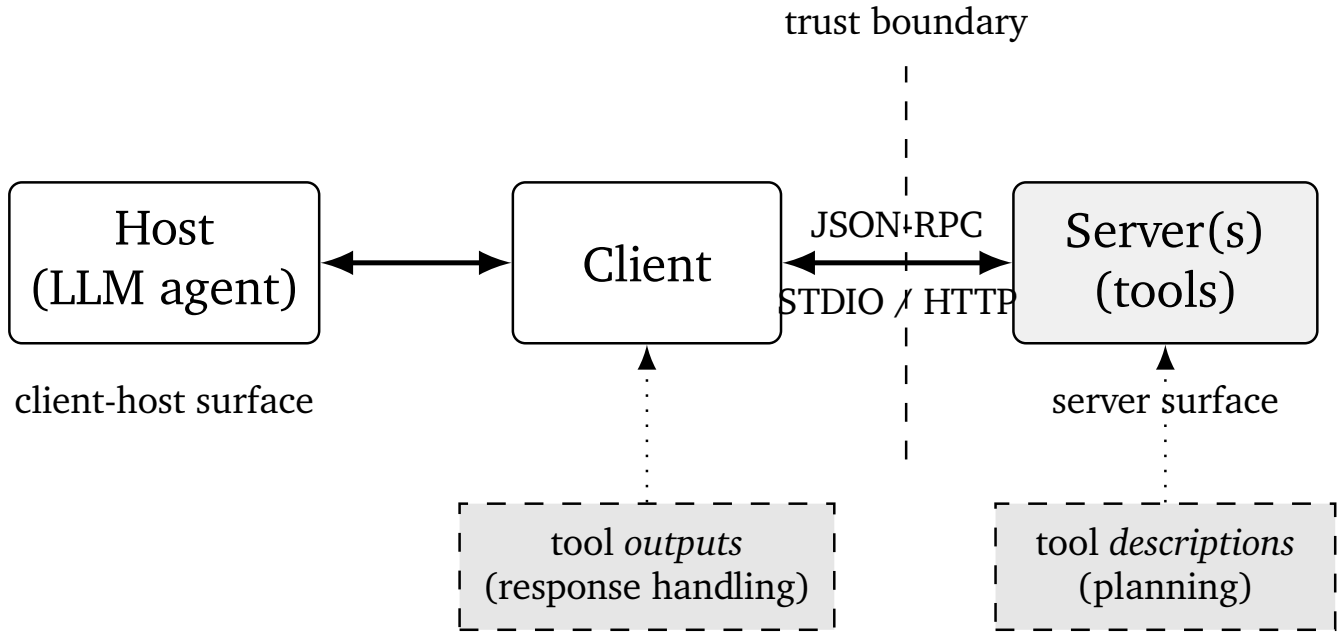
We scope the survey to the agent-to-Web3 attack surface and its mitigation. The distinct problem of agents as autonomous *generators* of exploits, where the agent is the attacker rather than the victim or conduit, is out of scope (Section 7) [Gervais and Zhou \(2025\)](#).

## 2. Background

This section establishes the protocol mechanics, the surrounding agent stack, and the key-custody and identity primitives that the rest of the paper analyzes. Readers familiar with MCP may skip 2.1 – 2.2.

### 2.1. Tool calling, MCP, and skills

Three mechanisms enable a language model to act beyond text generation, each with distinct trust profiles. *Tool calling* (or function calling) lets a model invoke a developer-defined function through a structured call.

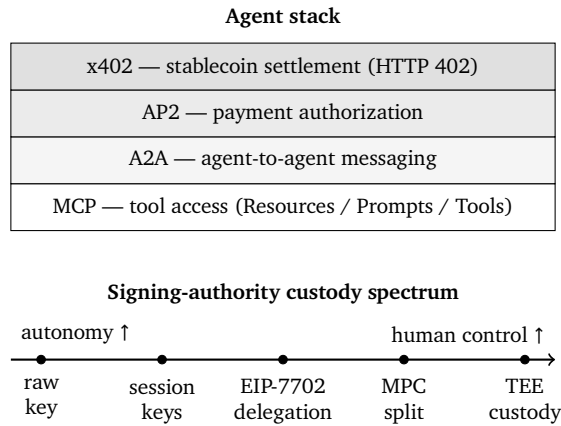


**Figure 2:** MCP trust boundaries and the two untrusted-content entry points. Tool descriptions enter at planning and tool outputs at response handling. Both cross into the agent’s context as trusted input: “the trust boundary is the schema” (Sec. 2). The four surfaces of Table 1 map onto this dataflow.

It emerged from work on reasoning-and-acting agents and tool-augmented models Yao et al. (2023), Schick et al. (2023), Patil et al. (2023). MCP standardizes this across providers, defining a common interface so that any compliant client can use any compliant tool server Hou et al. (2025). *Skills* (or plugins) package tools, instructions, and resources into installable units. The trust profiles differ. A tool called trusts the developer who wrote the function. MCP additionally trusts whoever operates the server and authored its tool descriptions. A skill trusts whoever published the package. Each added layer of indirection is an additional party to trust and an additional attack surface, as shown later.

## 2.2. MCP mechanics

MCP follows a “client host server” model over JSON-RPC Anthropic (2024). A host application embeds a client, which connects to one or more servers that expose tools. Client and server negotiate capabilities at connection time, and the host’s model selects and invokes tools over the session. The protocol exposes several primitives (e.g., resources, prompts, and tools, with later additions for sampling and elicitation) and supports both local (STDIO) and networked (streamable HTTP) transports Hou et al. (2025). The most security-critical mechanism here is that a tool’s *description*, the natural-language metadata that declares what the tool does and how to call it, is supplied to the model as trusted context at planning time. The model has no native means to distinguish a description authored by an honest server from one authored by an adversary. In MCP, the trust boundary is the schema. This single design fact seeds the largest class of protocol-level attacks in Section 4. Figure 2 makes the boundary concrete.



**Figure 3:** The agent stack (top) and the signing-authority custody spectrum (bottom). MCP at the tool layer is where the attacks of Table 1 land. The custody spectrum, from raw keys to TEE-isolated keys, governs the blast radius of the signing-authority amplifier.

### 2.3. The agent stack and Web3

MCP rarely operates alone. A surrounding stack has emerged. The Agent2Agent (A2A) protocol carries messages between agents [Google \(2025\)](#). The Agent Payments Protocol (AP2) standardizes the process by which an agent obtains authorization to pay [Stavan Parikh \(2025\)](#). And x402 revives the HTTP 402 status code to settle stablecoin payments over a web request [Erik et al. \(2025\)](#). The upper half of Figure 3 shows the layering: MCP grants tool access, A2A carries inter-agent messages, AP2 authorizes payment, and x402 settles it. The stack matters for this survey because it is where an agent acquires *payment and signing authority*, the capability governed by the next subsection’s custody models.

### 2.4. Agent-with-wallet key custody

When an agent transacts on a blockchain, the central design question is how it holds signing authority. The options form a spectrum from least to most constrained (lower half of Figure 3). At one extreme, an agent holds a *raw private key*, granting unrestricted signing: operationally simple and maximally dangerous. *Session keys* limit a key’s scope or lifetime. EIP-7702, shipped in Ethereum’s 2025 Pectra upgrade, lets an externally owned account delegate to contract code, making the scope of an agent’s signing authority programmable. If the delegation is broad, then it will be broadly dangerous [Vitalik et al. \(2024\)](#). *Multi-party computation (MPC)* splits signing authority so no single party (including the agent) can sign alone [Alqithami \(2026\)](#), [Staley \(2026\)](#), [Cobo \(2026\)](#). *Trusted execution environments (TEEs)* isolate keys in attested hardware [Ménétrey et al. \(2022\)](#), [Shepherd et al. \(2018\)](#). We define these here once. Section 3 treats them as deployment patterns and Section 6 as defenses.

### 2.5. Agent identity and authorization

Distinct from *what an agent can sign* is *who an agent is* and *what it is permitted to do*. Access to MCP servers is increasingly mediated by OAuth 2.1. Inter-agent identity is asserted through signed agent cards or decentralized identifiers, and transport-level authentication uses mutual TLS [GJ \(2026\)](#), [Raj \(2026\)](#), [Chang et al. \(2025\)](#), [W3C \(2022\)](#), [OWASP Foundation \(2026\)](#). These matter to this survey for a specific reason. Identity is simultaneously an *attack surface* (unsigned or replayable identity metadata enables impersonation

(Section 4)) and a *defense* (signed cards and mutual TLS are among the mitigations of Section 6).

### 3. How Agents Use MCP on Web3

Section 2 described the primitives. This section describes how they are actually deployed, and the design patterns and anti-patterns that shape the resulting attack surface.

#### 3.1. Integration modes

The first thing that shapes the attack surface is how deeply an agent is wired into the chain. Agent interaction with a blockchain falls into three modes. On the *read path*, the agent queries on-chain state (e.g., balances, prices, contract state) through RPC-backed tools. This is the lowest-risk mode, as no signing is involved. On the *write path*, the agent builds, signs, and submits transactions. This is where the signing authority (Section 2.4) is exercised and where the consequences of a compromised tool call become irreversible. The third mode is *machine-to-machine payment*, in which agents pay for resources or each other's services through x402-style settlement Erik et al. (2025). Production MCP servers already span all three modes, from general multi-chain servers to exchange-operated trading integrations strangelove ventures (2025), JamesANZ (2026), BanklessAPI (2025), AbdelStark (2025), Bybit (2026).

#### 3.2. Design patterns

Several recurring patterns shape how agents are deployed against Web3, and some double as mitigation. A *stateless per-request server* holds no signing authority between calls, narrowing the window of exposure. *Multi-chain tool-gating* restricts which chains and which operations a given tool may touch, an application of least privilege that also limits blast radius. *Split-authority custody* (MPC) and *policy-constrained wallets* (wallets that enforce per-transaction limits or allow-lists) keep an agent from unilaterally moving arbitrary value. We note these here as deployment patterns. Section 6 returns to the gateway/proxy pattern as a defense in its own right.

#### 3.3. Anti-patterns

The corresponding anti-patterns recur across deployments and map directly to the attack classes of Section 4. *Over-permissioning* (e.g., granting an agent broad, standing signing authority rather than scoped, revocable authority) is the most consequential. An industry analysis of MCP implementations found injection-class weaknesses in a large fraction of deployments Endor Labs (2026). The *sanitization gap* (trusting tool inputs and outputs on the local STDIO channel without validation) underlies the command-injection class. And *plugin-as-trust-decision* (installing a skill or server without treating it as the security decision it is) is what makes supply-chain compromise effective. The same marketplace dynamics that grew one skill registry past 10,700 entries also carried hundreds of malicious skills Alex, Oren Yomtov (2026).

## 4. An Attack-Surface Taxonomy for Agent Tool-Use

The security literature on the Model Context Protocol has grown quickly but unevenly: individual disclosures, benchmark papers, and vendor analyses each describe attacks against a different part of the protocol, using incompatible vocabularies. To reason about which of these attacks matter when an agent acts on a blockchain, they must first be organized. Throughout this paper, we assume an adversary who can author, operate,

interpose on, or plant content reachable by the agent, but who cannot break standard cryptographic primitives or read keys in properly isolated custody (MPC or TEE). The adversary’s goal is to induce an action favoring the attacker, such as an unauthorized signed transaction or a leak of sensitive context. The agent is thus the victim or unwitting conduit, never the attacker, with the inverse case (the agent generating exploits) a distinct model excluded in Section 7. This section adopts two axes from prior systematization as organizing scaffolding and uses them to locate each known attack class. The Web3-specific analysis enabled by this scaffolding is deferred to Section 5.

#### 4.1. Organizing axes

We organize the attack surface along two axes. The first, the *surface* axis, follows MCPSecBench Yang et al. (2026) in distinguishing where an attack enters: the **user** (the human-supplied request and approvals), the **client-host** (the agent runtime and its model), the **transport** (the JSON-RPC channel between client and server), and the **server** (the tool provider). The second, the *lifecycle-stage* axis, follows MSB Zhang et al. (2025) in distinguishing when an attack acts within the MCP tool-use pipeline: **task planning** (when the agent decides which tool to call), **tool invocation** (when the call executes), and **response handling** (when the tool’s output re-enters the agent’s context).

Cutting across both axes is a distinction between **protocol-level** weaknesses, which arise from the MCP design itself (most importantly, its treatment of tool descriptions and tool outputs as trusted context) and **implementation-level** weaknesses, which arise from how a particular server or client is built. The distinction matters because the two classes demand different defenses: a protocol-level weakness recurs across every conforming implementation, whereas an implementation-level weakness can be patched in a single codebase. We map the resulting categories to the established OWASP and MAESTRO agent-threat vocabularies Maloyan and Namiot (2026) where they align, so that the taxonomy can be read alongside existing practitioner frameworks OWASP Foundation (2025).

Table 1 presents the taxonomy. Each row is an attack class, located on both axes and labeled protocol- or implementation-level, with a representative confirmed vulnerability or documented incident. We populate the representative column only with vulnerabilities that carry a confirmed identifier or a publicly documented incident, so that the taxonomy is anchored to demonstrated rather than hypothetical attacks.

#### 4.2. Attack classes

We describe each class briefly. Table 1 gives the full mapping.

**Tool poisoning.** The defining protocol-level weakness of MCP is that a tool’s natural-language description is injected into the agent’s context and treated as trusted. An adversary who controls a tool description can therefore embed instructions for the model to follow at planning time. The attack was first demonstrated by Invariant Labs Invariant Labs (2025) and has since been shown to be highly effective at scale. The MCPTox benchmark reports a 72.8% attack-success rate against a leading model, with refusal rates below 3% even for safety-tuned models Wang et al. (2025), indicating that model alignment alone does not mitigate the attack. A configuration-swap variant against a popular MCP client was assigned CVE-2025-54136 (CVSS 7.2) Check Point Research (2025).

**Indirect prompt injection.** Distinct from a poisoned tool description, indirect prompt injection arrives in the *content* an agent retrieves (e.g., a web page, a file, an email) and is acted upon when that content re-enters the model’s context at response handling. The general class predates MCP Greshake et al. (2023), but MCP broadens it by making external content trivially reachable through tools. A documented instance



**Table 1:** Attack-Surface Taxonomy for Agent Tool-Use Against Web3. Axes adopted from MCPSecBench (surface) and MSB (stage) as organizing scaffolding. Each class carries a representative confirmed CVE or documented incident.

| # | Attack class                          | Surface     | Stage             | Layer | Representative CVE / incident (verified)                              |
|---|---------------------------------------|-------------|-------------------|-------|-----------------------------------------------------------------------|
| 1 | Tool poisoning                        | server      | task planning     | P     | CVE-2025-54136 <i>MCPoison</i> (CVSS 7.2); MCPTox 72.8% ASR           |
| 2 | Indirect prompt injection             | client-host | response handling | P     | CVE-2025-54135 <i>CurXecute</i>                                       |
| 3 | Command / STDIO injection             | transport   | tool invocation   | I     | CVE-2025-6514 <i>mcp-remote</i> (9.6); CVE-2026-30615 <i>Windsurf</i> |
| 4 | RCE via reference-impl. flaw          | server      | tool invocation   | I     | CVE-2025-68143/68144 <i>mcp-server-git</i> (8.8/7.1)                  |
| 5 | Sandbox / path-containment escape     | server      | tool invocation   | I     | CVE-2025-53109/53110 <i>EscapeRoute</i> (8.4/7.3)                     |
| 6 | Auth / authorization gap              | client-host | tool invocation   | I     | CVE-2025-49596 <i>MCP Inspector</i> (9.4)                             |
| 7 | Supply-chain (malicious server/skill) | server      | task planning     | I     | <i>postmark-mcp</i> (first documented); 341→824 malicious skills      |
| 8 | Preference manipulation               | server      | task planning     | P     | MPMA preference-manipulation                                          |
| 9 | Identity / impersonation              | transport   | task planning     | P     | A2A Agent-Card tampering / replay                                     |

against a popular agentic IDE was assigned CVE-2025-54135 [NIST \(2025\)](#).

**Command and STDIO injection.** At the transport layer, MCP servers that pass arguments to a shell or that trust the local STDIO channel are exposed to command injection. The most severe confirmed example, in the widely used ‘mcp-remote’ connector, was assigned CVE-2025-6514 with a CVSS score of 9.6 [Or Peles \(2025\)](#). A zero-click prompt-injection-to-code-execution chain in another agentic IDE was assigned CVE-2026-30615 [Moshe Siman Tov Bustan \(2026\)](#).

**Remote code execution via reference-implementation flaws.** Several of the official MCP reference servers shipped exploitable code. The Git server carried an arbitrary-path and argument-injection pair, CVE-2025-68143 (CVSS 8.8) and CVE-2025-68144 (7.1) [Lakshmanan \(2026\)](#). These are implementation-level. They were fixed in the affected servers, but their prevalence in *reference* code meant the flaws propagated into many downstream deployments.

**Sandbox and path-containment escape.** The File system reference server permitted symlink- and containment-based escapes from its intended directory, assigned CVE-2025-53109 (CVSS 8.4) and CVE-2025-53110 (7.3) [Elad Beber \(2025\)](#). The recurrence of containment bypasses across implementations suggests the sandboxing model, not any single bug, is the weak point.

**Authentication and authorization gaps.** Tooling around MCP frequently ships open by default. The MCP Inspector debugging tool exposed an unauthenticated interface allowing remote code execution, assigned CVE-2025-49596 with a CVSS score of 9.4 [Avi Lumelsky \(2025\)](#).

**Supply-chain compromise.** Because MCP servers and agent skills are distributed through open registries with low barriers to publication, they inherit the supply-chain risks of any package ecosystem. The first publicly documented malicious MCP server, a counterfeit of a legitimate email connector, added a single line that blind-copied every sent message to an attacker-controlled address. It was downloaded approximately 1,500 times before removal [Idan Dardikman \(2025\)](#). At the skill layer, a coordinated campaign saw the count of malicious skills in one marketplace rise from 341 to 824 as the marketplace itself grew past 10,700

entries [Alex, Oren Yomtov \(2026\)](#).

**Preference manipulation.** Beyond injecting instructions, an adversary can manipulate *which* tool the agent prefers, steering it toward an attacker-controlled tool through crafted metadata. This preference-manipulation attack operates at planning time and largely evades per-call output checks [Wang et al. \(2026\)](#).

**Identity and impersonation.** At the multi-agent layer of the stack (Section 2.3), an agent’s identity is asserted through metadata such as agent cards. When unsigned, they can be tampered with or replayed to impersonate a trusted agent. This surface is documented in analyses of the A2A layer [Raj \(2026\)](#) and becomes consequential in Web3 precisely because an impersonated agent may hold signing authority.

Across all classes, an independent measurement of prevalence is provided by an industry analysis of 2,614 MCP implementations, which reported path-traversal, code-injection, and command-injection weaknesses in 82%, 67%, and 34% of implementations, respectively [Endor Labs \(2026\)](#).

## 5. Why Web3 Changes the Threat Model: Four Amplifiers and a Risk Map

The taxonomy of Section 4 is not specific to blockchains. The same attack classes apply wherever agents call tools. What changes on Web3 is not the attacks but their *consequences*. This section argues that four properties of the blockchain execution layer systematically amplify the impact of agent tool-use attacks, turning recoverable failures into irreversible loss, and then maps each attack class to its amplified Web3 impact in a risk-mapping matrix (Table 2), the paper’s central contribution.

### 5.1. Four amplifiers

We identify four amplifiers. Each is a property of the Web3 setting, not of MCP, and each independently worsens the consequence of an attack that Section 4 catalogs.

**Irreversibility.** A settled blockchain transaction cannot be reversed, charged back, or undone by an intermediary. Finality is the defining property of the settlement layer. In conventional software, an attacker who induces a wrong action can often be remediated: a transfer reversed, a session revoked, a backup restored. On-chain, the same induced action is permanent. Irreversibility is therefore the amplifier that converts every other impact in the matrix from a recoverable error into a standing loss. It is the unifying claim of this section.

**Signing authority.** An agent that holds, or can invoke, signing authority can move value directly: a single signed transaction is itself the harmful act, with no further step required. The progression of key-custody models in Section 2.4 (from raw keys through scoped delegation under EIP-7702 to split-authority and hardware-isolated custody) exists precisely because granting an automated party signing authority is dangerous. EIP-7702’s delegation semantics make the scope of that authority explicit and, when misconfigured, broad [Vitalik et al. \(2024\)](#). The amplifier is that in Web3, the blast radius of a compromised agent is financial by default.

**Continuous autonomy.** Agents operate continuously, ingest untrusted external content, and increasingly act without per-step human review. This is the amplifier behind the action-tool shift noted in Section 1: the rise from 27% to 65% of tool use [Stein \(2026\)](#). The window between a successful injection and an irreversible on-chain action is bounded only by the agent’s own latency, not by a human’s.

**Sequence-level composition.** The fourth amplifier is the most subtle. Individually authorized tool calls can be composed into a sequence whose aggregate effect is malicious (e.g., a permitted read, a permitted



computation, and a permitted transfer chaining into an exfiltration or drain pipeline) even though no single call violates a policy. Multi-turn evaluation shows that agents are substantially more vulnerable when attacks span several interactions than when judged call-by-call [Zong et al. \(2025\)](#), and documented sequence-level chains (authorized tool calls that combined into exfiltration pipelines) appear in incident analysis [Moshe Siman Tov Bustan \(2026\)](#). Static, per-call defenses are, by construction, blind to the composition.

These amplifiers are not independent in effect: irreversibility sets the stakes, signing authority and continuous autonomy determine how fast and how far an attack propagates, and sequence-level composition determines how easily it evades per-call defenses. Their conjunction is what distinguishes Web3 agent risk from generic agent risk.

## 5.2. Mechanism

The cleanest demonstrated mechanism connecting an agent tool call to direct value movement is the malicious-router study, which examined intermediaries sitting between an agent and its model. Of 428 routers tested, 26 were found to inject malicious tool calls or access planted credentials, and the authors show that a single rewritten tool call is sufficient for arbitrary code execution [Liu et al. \(2026\)](#). The authors further report a client-wallet drain of roughly half a million dollars. We note this figure with caution, as the reproducible demonstration involved minimal funding and the amount has not been independently confirmed. The mechanism (one rewritten call, executed with signing authority, settled irreversibly) is what matters here, and it holds regardless of the disputed figure.

## 5.3. The risk-mapping matrix

Table 2 maps each attack class from the taxonomy to its dominant Web3 impact, the amplifier or amplifiers that magnify it, a representative mitigation (developed in Section 6), and the residual gap that the mitigation does not close.

Three observations follow from the matrix. First, the impact of every row is qualitatively worsened by irreversibility. The same tool-poisoning or injection attack that yields a recoverable error off-chain yields a standing loss on-chain. Second, sequence-level composition recurs across the tool-poisoning, indirect-injection, supply-chain, and preference-manipulation rows. It is the amplifier least addressed by current defenses, because those defenses operate per call. Third, and most consequential for the research agenda, several rows have no adequate mitigation measures. The matrix’s blank and partial mitigation cells are not omissions but findings, and they define the open problems of Section 7.

# 6. Defenses and Their Limits

The mitigation column of Table 2 points to the defenses surveyed here. We organize them as families mapped to the attack surface, treat blockchain-based defenses at moderate depth, and close with an honest account of what current defenses do not cover.

## 6.1. Defense families

**Defense-in-depth and gateways.** The most widely advocated posture is layered defense around the MCP boundary [Xing et al. \(2025\)](#), [Narajala and Habler \(2025\)](#). Its most concrete form is a *gateway* or proxy that sits between client and servers and inspects traffic at runtime (e.g., validating tool descriptions, filtering tool

**Table 2:** Web3 Risk-Mapping Matrix. Amplifiers: Irr=irreversibility, Sig=signing authority, Aut=continuous autonomy, Seq=sequence-level composition. Blank/partial mitigations denote open problems (Sec. 7).

| # | Attack class              | Web3 impact                        | Amplifiers | Sev.   | Mitigation (Sec. 6)                | Residual gap                                   |
|---|---------------------------|------------------------------------|------------|--------|------------------------------------|------------------------------------------------|
| 1 | Tool poisoning            | unauthorized tx; irreversible loss | Irr, Seq   | Sig, H | Signed/immutable manifests (ETDI)  | semantic poisoning of signed tool; refusal <3% |
| 2 | Indirect prompt injection | unauthorized tx; data exfil        | Aut, Seq   | H      | Gateway inspection; HITL on writes | agent acts on consumed content by design       |
| 3 | Command / STDIO injection | irreversible loss; key compromise  | Sig        | H      | Transport hardening; sandboxing    | impl. coverage uneven; protections <30%        |
| 4 | RCE via ref-impl flaw     | key compromise; irreversible loss  | Sig        | H      | Patching; least-privilege exec     | zero-day window vs 24/7 autonomy               |
| 5 | Sandbox escape            | data exfil; key compromise         | Sig        | M-H    | Hardened sandbox; path-allowlist   | containment bypasses recur                     |
| 6 | Auth gap                  | unauthorized tx; key compromise    | Sig        | H      | OAuth 2.1; mTLS; no default-open   | misconfiguration prevalent                     |
| 7 | Supply-chain              | data exfil; irreversible loss      | Aut, Sig   | H      | Registry provenance; signing       | low publish barrier; trust-on-first-use        |
| 8 | Preference manipulation   | unauthorized tx                    | Seq, Aut   | M      | Tool-selection policy; cross-check | subtle steering evades per-call checks         |
| 9 | Identity / impersonation  | unauthorized tx                    | Sig, Seq   | M      | Signed Agent Cards; mTLS           | delegation-chain identity unsolved             |

outputs, and enforcing policy on invocations [Kumar et al. \(2025\)](#), [Boyu Wang \(2026\)](#)). A gateway is the natural enforcement point because it is the one place that sees every call. It counters injection and poisoning at the moment of use rather than relying on the model to resist them.

**Provenance and signing.** Tool poisoning and supply-chain attacks both exploit the mutability and unverified origin of tool definitions. Signing them at the source addresses the root cause: the Enhanced Tool Definition Interface (ETDI) binds tool definitions to OAuth-backed cryptographic identities with immutable, versioned manifests, so that a tampered or silently-updated tool fails verification [Bhatt et al. \(2025\)](#). Manifest signing and registry provenance generalize this approach [Jamshidi et al. \(2026\)](#).

**Custody as defense.** The key-custody spectrum of Section 2.4 is itself a defense gradient: scoped delegation under EIP-7702, MPC split-authority, and TEE-isolated keys each shrink the blast radius of a compromised agent by constraining what it can sign. Policy-constrained wallets add per-transaction limits and allowlists, so that even a successful injection cannot move an arbitrary value. These directly blunt the signing authority amplifier.

**Identity.** Against the impersonation surface of Section 2.5, signed agent cards and mutual TLS authenticate which agent is which, raising the bar for replay and impersonation at the inter-agent layer [Raj \(2026\)](#).

## 6.2. Blockchain-based defenses

A distinct line of work inverts the paper’s direction, using blockchain mechanisms to secure the agent-tool ecosystem itself. Three categories recur. *On-chain registries* record signed tool provenance and revocation as a tamper-evident state. *Attested execution* pairs TEEs with on-chain attestation so that a server can prove what code it ran. *Crypto-economic trust* has a tool or agent operators stake value that is slashable on misbehavior.

Decentralized MCP architectures combine several of these [demcp \(2026\)](#). These approaches are early and carry open questions (e.g., registry governance, attestation cost, and the circularity of using one trust system to bootstrap another), and we present them as a promising but unsettled direction rather than a solution.

### 6.3. What current defenses do not cover

The honest assessment is that defenses are improving but remain insufficient, and the gaps are structural rather than incidental.

First, *measured effectiveness is low*. A systematic benchmark found that existing protections stop fewer than 30% of attacks on average [Yang et al. \(2026\)](#), and model-level safety is weaker still. Refusal rates against tool-poisoning fall below 3% even for safety-tuned models [Wang et al. \(2025\)](#). Alignment alone does not defend the tool layer.

Second, *the core weakness is not fully fixable at the protocol layer*. An agent that acts on the content it consumes will act on malicious content that reaches it. This is a consequence of the design, not a bug to be patched. A vendor characterization of one such report as expected behavior rather than a vulnerability illustrates the point. The boundary between feature and flaw is itself contested [Moshe Siman Tov Bustan \(2026\)](#).

Third, *per-call defenses miss sequence-level composition*. Gateways and policy checks evaluate calls individually, but the sequence-level composition amplifier (Section 5) produces harm from an aggregate of individually permitted calls. No widely-deployed defense reasons over the sequence. This gap, together with the unsolved problem of delegation-chain identity, defines much of the research agenda in Section 7.

## 7. Related Work and Open Problems

### 7.1. Positioning

Several recent surveys touch on the agent–blockchain intersection, but each addresses a different question than this one.

The closest in subject is the systematization of AI agents for blockchain by Romandini et al. [Romandini et al. \(2025\)](#), which surveys how agents *assist* blockchain work: analyzing on-chain data, optimizing transaction strategies, and detecting smart-contract vulnerabilities. Our direction is the opposite. We study the attack surface that opens when agents *act on* Web3 through tool calling. Their agents help secure the chain, while ours are the thing to be secured.

A broader interoperability survey by Alqithami [Alqithami \(2026\)](#) reviews 317 works on agent–blockchain integration and contributes a five-part taxonomy of integration patterns and a threat model for agent-driven transactions. Its scope overlaps ours more than the others, but its lens is *standards and execution models*. Ours is the security amplification specific to the execution layer, expressed through the four-amplifier framework and the risk-mapping matrix. We differentiate on analytical framing and the defense-coverage artifact, not on cataloging integration patterns.

A systematization of the MCP security ecosystem [Gaire et al. \(2025\)](#) catalogs MCP threats and defenses without a Web3 focus. Our work can be considered as extending it with the blockchain execution-layer overlay that makes the attack surface qualitatively different. Table 3 situates this survey against the verified MCP-security benchmarks. Each measures attack success, but none carries a Web3 execution-layer lens.

**Table 3:** MCP-Security Benchmarks and This Survey. Only independently verified benchmarks are listed. This work adds the Web3 execution-layer overlay absent from all prior benchmarks.

| Work               | Axis            | # at-tacks | Web3-specific?                          |
|--------------------|-----------------|------------|-----------------------------------------|
| MCPSecBench        | 4 sur-faces     | 17 types   | no; protect. <30%                       |
| MSB                | 3 stages        | 12 types   | no (ASR 40.71%)                         |
| MCPTox             | tool de-scr.    | —          | no (72.8% ASR)                          |
| <b>This survey</b> | surface × stage | 9 classes  | <b>yes (4 amplifiers + risk matrix)</b> |

A general agent-security systematization [Dehghantanha and Homayoun \(2026\)](#) provides the broader backdrop.

## 7.2. Scope boundary

One adjacent problem is deliberately out of scope. The use of agents as autonomous *generators* of exploits inverts the threat model (the agent is the attacker, not the victim or conduit) and is studied separately [Gervais and Zhou \(2025\)](#). We exclude it to keep the usage-to-risk direction sharp.

## 7.3. From gaps to directions

The blank and partial mitigation cells of Table 2, together with the structural defense gaps of Section 6 are not loose ends but a structured agenda. Section 8 develops the five research directions they imply.

# 8. Research Directions

The preceding analysis is not only descriptive: the risk-mapping matrix (Table 2) and the defense synthesis (Section 6) together expose where the field’s hardest unsolved problems lie. Each of the four amplifiers generates a distinct research challenge that current techniques do not address, and a fifth, cross-cutting problem concerns how agent-tool security can be evaluated at all. We frame five directions, each with a technical obstacle, an explanation of why current approaches fall short, and concrete research questions. The directions are grounded in the survey’s verified findings rather than offered as open-ended speculation.

## 8.1. Semantic integrity of tool descriptions

**Obstacle.** Because an MCP tool’s natural-language description is supplied to the agent as trusted context (Section 2), the description is the primary protocol-level attack surface (Section 4). The unsolved problem is verifying not who authored a description but whether it is *honest*: whether a tool’s declared behavior matches its actual behavior.

**Why current approaches fall short.** Provenance and signing schemes such as ETDI bind a description to a cryptographic identity, but a validly signed tool can still carry a poisoned description. Signing proves origin, not intent. Model-level alignment is weaker still (refusal rates against tool-poisoning fall below 3% even

for safety-tuned models Wang et al. (2025)), and gateway scanning detects surface patterns, not semantic deception. None of these verifies meaning.

#### Research questions.

- Can a tool’s description be formally checked for conformance against its observed input–output behavior, turning “description integrity” into a runtime contract rather than a publish-time signature?
- What is the detection ceiling for semantic poisoning that uses no known trigger tokens and remains consistent with the tool’s nominal function?
- Can agents be trained or instrumented to treat tool descriptions as untrusted input without losing the usability that makes MCP valuable?

### 8.2. Sequence-level defense

**Obstacle.** The sequence-level composition amplifier (Section 5) produces harm from an aggregate of individually authorized calls (e.g., a permitted read, a permitted computation, and a permitted transfer chaining into a drain or exfiltration pipeline) where no single call violates a policy. The harmful object is the trajectory, not any call within it.

**Why current approaches fall short.** Gateways and policy engines evaluate calls in isolation by construction. No widely deployed defense reasons over a sequence (Section 6). This is the amplifier least covered by current mitigations. It appears across the tool-poisoning, indirect-injection, supply-chain, and preference-manipulation rows of Table 2, yet every listed mitigation is per-call.

#### Research questions.

- Can a stateful monitor flag a dangerous sequence of permitted calls without an unacceptable false-positive rate on benign multi-step tasks?
- What is the minimal cross-call state required to detect canonical drain and exfiltration patterns?
- Can per-session capability budgets, for example, a cumulative value-at-risk bound across a trajectory, limit compositional harm without crippling legitimate automation?

### 8.3. The irreversibility and autonomy gap

**Obstacle.** Blockchain settlement is final, and agents act continuously without per-step human review. The window between a successful injection and an irreversible on-chain action is therefore bounded only by the agent’s own latency, not by a human’s. Two amplifiers (irreversibility and continuous autonomy) compound here: there is no human-speed window in which to intervene, and once a transaction settles, after-the-fact detection is moot.

**Why current approaches fall short.** Human-in-the-loop confirmation does not scale to continuous autonomous operation, and conventional incident response assumes an observable, reversible window that on-chain execution does not provide. The defenses of Section 6 reduce the probability of compromise but do not address what happens in the interval between compromise and irreversible settlement.

#### Research questions.

- Can high-stakes actions be reversibly staged (e.g., through commit–reveal, time-locks, or challenge periods) so that an irreversible action becomes interruptible without destroying agent utility?

- Which on-chain primitives (escrow, optimistic delay, circuit breakers) most effectively convert irreversible actions into recoverable ones for autonomous agents?
- How should an agent calibrate when to demand human confirmation as a function of value-at-risk and the trust level of the triggering input?

#### 8.4. Delegation-chain identity and accountability

**Obstacle.** When an agent acts on behalf of a user who, in turn, acts for an organization, both *authority* (what may this chain legitimately sign?) and *accountability* (who is responsible for a given action?) are poorly modeled. This is the matrix’s least-covered row (Table 2, identity): an impersonated or over-privileged agent in such a chain can exercise signing authority it was never meant to hold.

**Why current approaches fall short.** Signed Agent Cards and mutual TLS authenticate a single hop, not a chain of delegations. Scoped on-chain delegation, such as EIP-7702, governs what an account may sign but is not bound to the off-chain chain of principals behind the agent, so a compromised intermediate agent can act within its on-chain grant while exceeding its actual mandate.

##### Research questions.

- Can delegation be made verifiable end-to-end: a cryptographic chain linking a human principal through intermediary agents to an on-chain action?
- How should scoped signing authority be bound to a verified delegation chain so that a compromised intermediate agent cannot exceed its delegated grant?
- What revocation model is appropriate when one link in a live delegation chain is found to be compromised?

#### 8.5. Evaluation and assurance under non-determinism

**Obstacle.** Agent behavior is non-deterministic, which frustrates the test-and-certify model that secures conventional software. At present, the field can benchmark the average failure rate of an agent-tool deployment, but cannot certify its safety.

**Why current approaches fall short.** Existing benchmarks measure average attack-success rate on fixed attack sets, protections stop fewer than 30% of attacks on average [Yang et al. \(2026\)](#), but they neither bound worst-case behavior nor generalize to unseen attacks. Compounding this, the boundary between a vulnerability and intended behavior is itself contested: when an agent acting on consumed content is exploited, vendors may classify the report as expected behavior rather than a flaw [Moshe Siman Tov Bustan \(2026\)](#), leaving disclosure and response norms undefined.

##### Research questions.

- What would a certification regime (as opposed to a benchmark) for agent-tool security look like: assurance cases, bounded guarantees, or worst-case rather than average-case metrics?
- Can a security metric be defined that bounds the worst-case behavior of a non-deterministic agent over a space of adversarial inputs?
- How should disclosure and incident-response norms be standardized for an ecosystem in which the exploited behavior is frequently the intended behavior?



## 8.6. Limitations

Three caveats bound the claims above. The survey is scoped to the usage-to-risk direction and treats defenses only insofar as they map to that surface. The risk-mapping matrix (Table 2) assigns qualitative severity rather than measured scores, and the CVSS values we cite are vendor-reported, with the one quantitative incident figure (Section 5.2) explicitly hedged. Finally, the MCP-security literature moves faster than any survey can track, so we anchor every cell to a confirmed identifier or documented incident, keeping the scaffolding usable as rows are superseded.

## 9. Conclusion

As agents move from reading to acting, the tools that grant them authority have become the attack surface. This survey has argued that when that authority is exercised on Web3, the blockchain execution layer turns the recoverable failures of generic agent security into standing, irreversible loss, through four amplifiers. The attack-surface taxonomy and the risk-mapping matrix organize this argument into durable artifacts: the taxonomy locates the known attack classes, and the matrix ties each to its amplified Web3 impact and to the defenses that do, and do not, cover it. Those defenses remain insufficient, and their gaps are structural: semantic tool integrity, sequence-level reasoning, delegation-chain identity, and safe interruption of irreversible action. The ecosystem is being built now, while these problems remain open.

## References

- AbdelStark. Bitcoin & lightning network mcp server, 2025. URL <https://github.com/abdelstark/bitcoin-mcp>.
- Alex, Oren Yomtov. Clawhavoc: 341 malicious clawedbot skills found by the bot they were targeting, February 2026. URL <https://www.koi.ai/blog/clawhavoc-341-malicious-clawedbot-skills-found-by-the-bot-they-were-targeting>.
- Saad Alqithami. Autonomous agents on blockchains: Standards, execution models, and trust boundaries, 2026. URL <https://arxiv.org/abs/2601.04583>.
- Anthropic. Introducing the model context protocol, November 2024. URL <https://www.anthropic.com/news/model-context-protocol>.
- Avi Lumelsky. Critical rce vulnerability in anthropic mcp inspector: Cve-2025-49596, 2025. URL <https://www.oligo.security/blog/critical-rce-vulnerability-in-anthropic-mcp-inspector-cve-2025-49596>.
- BanklessAPI. Onchain mcp, 2025. URL <https://docs.bankless.com/bankless-api/other-services/onchain-mcp>.
- Manish Bhatt, Vineeth Sai Narajala, and Idan Habler. Etdi: Mitigating tool squatting and rug pull attacks in model context protocol (mcp) by using oauth-enhanced tool definitions and policy-based access control, 2025. URL <https://arxiv.org/abs/2506.01333>.
- Boyu Wang. Mcp tool poisoning: An attack on the channel the model trusts most, 2026. URL <https://www.truefoundry.com/blog/blog-mcp-tool-poisoning-gateway-defense>.
- Bybit. Bybit mcp server, April 2026. URL <https://github.com/bybit-exchange/trading-mcp>.
- Gaowei Chang, Eidan Lin, Chengxuan Yuan, Rizhao Cai, Binbin Chen, Xuan Xie, and Yin Zhang. Agent network protocol technical white paper, 2025. URL <https://arxiv.org/abs/2508.00007>.
- Check Point Research. Cve-2025-54136: Mcpoison cursor ide: Persistent code execution via mcp trust bypass, August 2025. URL <https://research.checkpoint.com/2025/cursor-vulnerability-mcpoison/>. CVE-2025-54136.
- Cobo. Cobo launches agentic wallet: Three industry firsts redefining how ai agents interact on-chain, 2026. URL <https://www.cobo.com/agentic-wallet/news/cobo-launches-agentic-wallet-how-ai-agents-interact-on-chain>.
- Ali Dehghantanha and Sajad Hodayoun. Sok: The attack surface of agentic ai – tools, and autonomy, 2026. URL <https://arxiv.org/abs/2603.22928>.
- demcp. Awesome web3 mcp servers, 2026. URL <https://github.com/demcp/awesome-web3-mcp-servers>.
- Elad Beber. Escaperoute: Breaking the scope of anthropic’s filesystem mcp server (cve-2025-53109 & cve-2025-53110), 2025. URL <https://cymulate.com/blog/cve-2025-53109-53110-escaperoute-anthropic/>.

- Endor Labs. Classic vulnerabilities meet ai infrastructure: Why mcp needs appsec, January 2026. URL <https://www.endorlabs.com/learn/classic-vulnerabilities-meet-ai-infrastructure-why-mcp-needs-appsec>.
- Reppel Erik, Caspers Ronnie, Leffew Kevin, Organ Danny, Kim Dan, and Dalal Nemil. x402: An open protocol for internet-native payments, 2025. URL <https://www.x402.org/x402-whitepaper.pdf>.
- Shiva Gaire, Srijan Gyawali, Saroj Mishra, Suman Niroula, Dilip Thakur, and Umesh Yadav. Systematization of knowledge: Security and safety in the model context protocol ecosystem, 2025. URL <https://arxiv.org/abs/2512.08290>.
- Arthur Gervais and Liyi Zhou. Ai agent smart contract exploit generation, 2025. URL <https://arxiv.org/abs/2507.05558>.
- GJ. Agent auth: Why oauth wasn't built for this. Apideck Blog, April 2026. URL <https://www.apideck.com/blog/agent-auth-oauth-ai-agents>. Updated Apr. 27, 2026.
- Google. A2A (Agent2Agent) protocol. <https://a2a-protocol.org>, 2025.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023. doi: 10.1145/3605764.3623985.
- Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (mcp): Landscape, security threats, and future research directions, 2025. URL <https://arxiv.org/abs/2503.23278>.
- Idan Dardikman. First malicious mcp in the wild: The postmark backdoor that's stealing your emails, September 2025. URL <https://www.koi.ai/blog/postmark-mcp-npm-malicious-backdoor-email-theft>.
- Invariant Labs. Mcp security notification: Tool poisoning attacks, April 2025. URL <https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks>.
- JamesANZ. Evm mcp server, 2026. URL <https://github.com/JamesANZ/evm-mcp>.
- Saeid Jamshidi, Kawser Wazed Nafi, Arghavan Moradi Dakhel, Foutse Khomh, Amin Nikanjam, and Mohammad Adnan Hamdaqa. Secure tool manifest and digital signing solution for verifiable mcp and llm pipelines, 2026. URL <https://arxiv.org/abs/2601.23132>.
- S. Kumar, A. Girdhar, R. Patil, and D. Tripathi. Mcp guardian: A security-first layer for safeguarding mcp-based ai systems, 2025. URL <https://arxiv.org/abs/2504.12757>.
- Ravie Lakshmanan. Three flaws in anthropic mcp git server enable file access and code execution, 2026. URL <https://thehackernews.com/2026/01/three-flaws-in-anthropic-mcp-git-server.html>.
- Linux Foundation. Linux foundation announces the formation of the agentic ai foundation, December 2025. URL <https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundation>.

- Hanzhi Liu, Chaofan Shou, Hongbo Wen, Yanju Chen, Ryan Jingyang Fang, and Yu Feng. Your agent is mine: Measuring malicious intermediary attacks on the llm supply chain, 2026. URL <https://arxiv.org/abs/2604.08407>.
- Narek Maloyan and Dmitry Namiot. Breaking the protocol: Security analysis of the model context protocol specification and prompt injection vulnerabilities in tool-integrated llm agents, 2026. URL <https://arxiv.org/abs/2601.17549>.
- Nir Zadok Moshe Siman Tov Bustan, Mustafa Naamnih. Mcp supply chain advisory: Rce vulnerabilities across the ai ecosystem, April 2026. URL <https://www.ox.security/blog/mcp-supply-chain-advisory-rce-vulnerabilities-across-the-ai-ecosystem/>.
- Jämes Ménétrey, Christian Göttel, Anum Khurshid, Marcelo Pasin, Pascal Felber, Valerio Schiavoni, and Shahid Raza. *Attestation Mechanisms for Trusted Execution Environments Demystified*, page 95–113. Springer International Publishing, 2022. ISBN 9783031160929. doi: 10.1007/978-3-031-16092-9\_7. URL [http://dx.doi.org/10.1007/978-3-031-16092-9\\_7](http://dx.doi.org/10.1007/978-3-031-16092-9_7).
- Vineeth Sai Narajala and Idan Habler. Enterprise-grade security for the model context protocol (mcp): Frameworks and mitigation strategies, 2025. URL <https://arxiv.org/abs/2504.08623>.
- NIST. Cve-2025-54135 detail, 2025. URL <https://nvd.nist.gov/vuln/detail/CVE-2025-54135>.
- Or Peles. Critical rce vulnerability in mcp-remote: Cve-2025-6514 threatens llm clients, 2025. URL <https://jfrog.com/blog/2025-6514-critical-mcp-remote-rce-vulnerability/>. CVE-2025-6514; CVSS 9.6; fixed in mcp-remote 0.1.16.
- OWASP Foundation. Owasp top 10 for llm applications, 2025. URL <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.
- OWASP Foundation. About the owasp foundation, 2026. URL <https://owasp.org/about/>.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis, 2023. URL <https://arxiv.org/abs/2305.15334>.
- Vivek Raj. A2a protocol security: Authenticating agent-to-agent communication. SecureW2 Blog, May 2026. URL <https://securew2.com/blog/a2a-protocol-security>. Accessed June 7, 2026.
- Nicolo Romandini, Carlo Mazzocca, Kai Otsuki, and Rebecca Montanari. Sok: Security and privacy of ai agents for blockchain, 2025. URL <https://arxiv.org/abs/2509.07131>.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023. URL <https://arxiv.org/abs/2302.04761>.
- Carlton Shepherd, Raja Naeem Akram, and Konstantinos Markantonakis. Remote credential management with mutual attestation for trusted execution environments. In *Proceedings of the 12th International Conference on Availability, Reliability and Security*, 2018. doi: 10.1145/3230833.3232795.
- Ian Staley. A reference architecture for ai agents on blockchain infrastructure: Identity, policy, payments, and custody as composable primitives. SSRN, 2026. URL <https://doi.org/10.2139/ssrn.6650658>. SSRN Scholarly Paper.

- Rao Surapaneni Stavan Parikh. Announcing agent payments protocol (ap2), September 2025. URL <https://cloud.google.com/blog/products/ai-machine-learning/announcing-agents-to-payments-ap2-protocol>.
- Merlin Stein. How are ai agents used? evidence from 177,000 mcp tools, 2026. URL <https://arxiv.org/abs/2603.23802>.
- strangelove ventures. Web3 mcp, 2025. URL <https://github.com/strangelove-ventures/web3-mcp>.
- Buterin Vitalik, Wilson Sam, Dietrichs Ansgar, and lightclient. Eip-7702: Set code for eoas. Ethereum Improvement Proposal, 2024. URL <https://eips.ethereum.org/EIPS/eip-7702>.
- W3C. Decentralized identifiers (DIDs) v1.0: Core architecture, data model, and representations. W3C Recommendation, 2022. URL <https://www.w3.org/TR/did-core/>.
- Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. Mcptox: A benchmark for tool poisoning attack on real-world mcp servers, 2025. URL <https://arxiv.org/abs/2508.14925>.
- Zihan Wang, Rui Zhang, Yu Liu, Wenshu Fan, Wenbo Jiang, Qingchuan Zhao, Hongwei Li, and Guowen Xu. Mpma: Preference manipulation attack against model context protocol. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 2026. doi: 10.1609/aaai.v40i22.40898. URL <https://arxiv.org/abs/2505.11154>.
- Wenpeng Xing, Zhonghao Qi, Yupeng Qin, Yilin Li, Caini Chang, Jiahui Yu, Changting Lin, Zhenzhen Xie, and Meng Han. Mcp-guard: A multi-stage defense-in-depth framework for securing model context protocol in agentic ai, 2025. URL <https://arxiv.org/abs/2508.10991>.
- Yixuan Yang, Cuifeng Gao, Daoyuan Wu, Yufan Chen, Yingjiu Li, and Shuai Wang. Mcpsecbench: A systematic security benchmark and playground for testing model context protocols, 2026. URL <https://arxiv.org/abs/2508.13220>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL <https://arxiv.org/abs/2210.03629>.
- Dongsun Zhang, Zekun Li, Xu Luo, Xuannan Liu, Peipei Li, and Wenjun Xu. Mcp security bench (msb): Benchmarking attacks against model context protocol in llm agents, 2025. URL <https://arxiv.org/abs/2510.15994>.
- Xuanjun Zong, Zhiqi Shen, Lei Wang, Yunshi Lan, and Chao Yang. Mcp-safetybench: A benchmark for safety evaluation of large language models with real-world mcp servers, 2025. URL <https://arxiv.org/abs/2512.15163>.